Analyse quantitative de SOLIBRA (SLBC.CI), 01/01/2023 - 15/10/2026

Yolou Ahoua Marius Tresor, Trader commodities and currency

16 octobre 2026

Optique du code

Le notebook charge les fichiers produits par la collecte : cours et volumes, journal des exclusions, calendrier, dividendes, événements et indice BRVM Composite. Il contrôle d'abord les données, puis sépare les rendements admissibles de ceux qui franchissent une lacune, une suspension ou le fractionnement. Il calcule ensuite la performance, le drawdown, la volatilité, la VaR et l'Expected Shortfall, les diagnostics statistiques, la robustesse et la comparaison à l'indice. Chaque étape affiche son effectif. Aucune donnée n'est comblée, interpolée ni ajustée.

Configuration

Les paramètres de l'étude sont regroupés dans une seule cellule : période, dates des événements, fenêtres de calcul et nombre de répliques du bootstrap.

```
import os, io, sys, json, zipfile, platform, warnings, datetime as dt
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
import scipy
from scipy import stats
import statsmodels
import statsmodels.api as sm
from statsmodels.stats.diagnostic import acorr_ljungbox, het_arch
from statsmodels.tsa.stattools import adfuller, kpss

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", message=".*kpss.*")

ENTREE = "BASE_SOLIBRA_SLBC_2023_2026.zip"
SORTIE = "resultats"
DEBUT, FIN = pd.Timestamp("2023-01-01"), pd.Timestamp("2026-10-15")
SPLIT_DATE = pd.Timestamp("2024-09-30")
SUSP_DEBUT, SUSP_FIN = pd.Timestamp("2024-04-02"), pd.Timestamp("2024-09-10")
SEGMENTS = {
    "S1 pré-suspension": (pd.Timestamp("2023-01-01"), pd.Timestamp("2024-03-31")),
    "S2 reprise pré-fractionnement": (pd.Timestamp("2024-09-11"), pd.Timestamp("2024-09-29")),
    "S3 post-fractionnement": (SPLIT_DATE, FIN),
}
NIVEAUX_VAR = [0.95, 0.99]
MIN_QUEUE = 30
FENETRES_VOL = [20, 60]
LAGS_LJUNG = [5, 10]
LAGS_ARCH = 5
BLOC = 5
B_BOOT = 2000
GRAINE = 20261015
TAUX_SANS_RISQUE = None
dpi_png = 300
sns.set_theme(style="whitegrid", context="paper")
np.random.seed(GRAINE)
os.makedirs(os.path.join(SORTIE, "tables"), exist_ok=True)
os.makedirs(os.path.join(SORTIE, "figures"), exist_ok=True)
os.makedirs(os.path.join(SORTIE, "journal"), exist_ok=True)
```

A. Objectif et périmètre

L'étude décrit le cours, le volume et le risque historique de SOLIBRA entre le 1er janvier 2023 et le 15 octobre 2026. Elle porte sur les données publiées par la source retenue, et non sur les cotations officielles de la BRVM.

B. Chargement des fichiers

Les fichiers sont lus selon leur contenu réel. Un fichier CSV renommé en .zip est accepté ; une archive corrompue ou tronquée provoque une erreur explicite au lieu d'un plantage incompréhensible.

```
def type_reel(chemin):
    with open(chemin, "rb") as f:
        tete = f.read(8)
    if tete[:4] == b"PK\x03\x04":
        return "zip"
    if tete[:5] == b"%PDF-":
        return "pdf"
    if tete.lstrip()[:1] == b"<":
        return "html"
    return "texte"

def trouver_entree(nom):
    for racine in [".", "..", os.path.dirname(os.path.abspath(__file__)) if "__file__" in globals() else
"."]:
        p = os.path.join(racine, nom)
        if os.path.exists(p):
            return p
    raise FileNotFoundError(f"{nom} introuvable (cherché dans le dossier courant et le dossier parent)")

def lire_membres(chemin):
    t = type_reel(chemin)
    if t == "texte":
        return {os.path.basename(chemin): open(chemin, "rb").read()}
    if t != "zip":
        raise ValueError(f"{chemin} : type réel « {t} » ; attendu CSV ou ZIP")
    try:
        z = zipfile.ZipFile(chemin)
    except zipfile.BadZipFile as e:
        raise ValueError(f"{chemin} : signature ZIP présente mais archive illisible ({e})")
    if z.testzip() is not None:
        raise ValueError(f"{chemin} : membre corrompu dans l'archive")
    return {os.path.basename(n): z.read(n) for n in z.namelist() if not n.endswith("/")}

FICHIERS_REQUIS = ["SOLIBRA_SLBC_2023_2026_ANALYSE.csv", "SOLIBRA_SLBC_2023_2026_BRUT.csv",
                   "JOURNAL_CORRECTIONS_EXCLUSIONS.csv", "CALENDRIER_COUVERTURE_2023_2026.csv",
                   "SOLIBRA_DIVIDENDES_2023_2026.csv", "EVENEMENTS_SOLIBRA_2023_2026.csv",
                   "METADONNEES_SERIE.json", "Indice_BRVMC.csv"]
if "google.colab" in sys.modules or os.path.exists("/content"):
    from google.colab import files as _colab_files
    print("Téléversez les fichiers d'entrée :", ", ".join(FICHIERS_REQUIS))
    membres = {os.path.basename(k): v for k, v in _colab_files.upload().items()}
else:
    membres = lire_membres(trouver_entree(ENTREE))
manquants = [f for f in FICHIERS_REQUIS if f not in membres]
if manquants:
    raise FileNotFoundError(f"Fichiers manquants : {manquants}")
print("Fichiers reconnus :", sorted(membres))

def csv_depuis(nom, **kw):
    if nom not in membres:
        raise FileNotFoundError(f"{nom} absent de l'entrée ; présents : {sorted(membres)}")
    return pd.read_csv(io.BytesIO(membres[nom]), **kw)

analyse = csv_depuis("SOLIBRA_SLBC_2023_2026_ANALYSE.csv", dtype=str, keep_default_na=False)
brut = csv_depuis("SOLIBRA_SLBC_2023_2026_BRUT.csv", dtype=str, keep_default_na=False)
journal_src = csv_depuis("JOURNAL_CORRECTIONS_EXCLUSIONS.csv", dtype=str, keep_default_na=False)
calendrier = csv_depuis("CALENDRIER_COUVERTURE_2023_2026.csv", dtype=str, keep_default_na=False)
dividendes = csv_depuis("SOLIBRA_DIVIDENDES_2023_2026.csv", dtype=str, keep_default_na=False)
evenements = csv_depuis("EVENEMENTS_SOLIBRA_2023_2026.csv", dtype=str, keep_default_na=False)
meta = json.loads(membres["METADONNEES_SERIE.json"].decode("utf-8"))
indice_brut = csv_depuis("Indice_BRVMC.csv", sep=";", dtype=str, encoding="utf-8-sig",
keep_default_na=False)
rapport_indice = json.loads(membres["indices_report.json"].decode("utf-8")) if "indices_report.json" in
membres else None

assert list(analyse.columns) == ["Date", "Close", "Volume"], f"colonnes inattendues :
{list(analyse.columns)}"
print(f"ANALYSE : {len(analyse)} lignes ; BRUT : {len(brut)} ; journal : {len(journal_src)} ; indice :
{len(indice_brut)} lignes")
```

```
print("Calendrier : colonnes", list(calendrier.columns))
print("Statut du calendrier :", "officiel" if "statut_seance_officiel" in calendrier.columns else "NON
officiel (aucune colonne de séance établie)")
```

```
Téléversez les fichiers d'entrée : SOLIBRA_SLBC_2023_2026_ANALYSE.csv, SOLIBRA_SLBC_2023_2026_BRUT.csv,
JOURNAL_CORRECTIONS_EXCLUSIONS.csv, CALENDRIER_COUVERTURE_2023_2026.csv,
SOLIBRA_DIVIDENDES_2023_2026.csv, EVENEMENTS_SOLIBRA_2023_2026.csv, METADONNEES_SERIE.json,
Indice_BRVMC.csv
Fichiers reconnus : ['CALENDRIER_COUVERTURE_2023_2026.csv', 'EVENEMENTS_SOLIBRA_2023_2026.csv',
'Indice_BRVMC.csv', 'JOURNAL_CORRECTIONS_EXCLUSIONS.csv', 'METADONNEES_SERIE.json',
'SOLIBRA_DIVIDENDES_2023_2026.csv', 'SOLIBRA_SLBC_2023_2026_ANALYSE.csv',
'SOLIBRA_SLBC_2023_2026_BRUT.csv']
ANALYSE : 721 lignes ; BRUT : 725 ; journal : 68 ; indice : 5042 lignes
Calendrier : colonnes ['Date', 'jour', 'jour_ouvre', 'statut']
Statut du calendrier : NON officiel (aucune colonne de séance établie)
```

C. Audit de qualité et couverture

Les dates, les nombres, les doublons, les valeurs manquantes et les volumes sont contrôlés. Les effectifs annuels et trimestriels sont donnés, ainsi que la part des jours ouvrés portant une observation.

```
obs = analyse.copy()
obs["Date"] = pd.to_datetime(obs.Date, format="%Y-%m-%d", errors="coerce")
obs["Close"] = pd.to_numeric(obs.Close, errors="coerce")
obs["Volume"] = pd.to_numeric(obs.Volume, errors="coerce")
obs = obs.sort_values("Date").reset_index(drop=True)

brut_idx = brut.copy()
brut_idx["Date"] = pd.to_datetime(brut_idx.Date, format="%Y-%m-%d", errors="coerce")
brut_idx["Volume_FCFA"] = pd.to_numeric(brut_idx.Volume_FCFA, errors="coerce")
obs = obs.merge(brut_idx[["Date", "Volume_FCFA", "statut_analyse"]], on="Date", how="left")

audit = {
    "lignes_ANALYSE": len(obs),
    "date_min": obs.Date.min().date().isoformat(),
    "date_max": obs.Date.max().date().isoformat(),
    "dates_non_parsees": int(obs.Date.isna().sum()),
    "doublons_de_date": int(obs.Date.duplicated().sum()),
    "close_manquants": int(obs.Close.isna().sum()),
    "volume_manquants": int(obs.Volume.isna().sum()),
    "close_non_positifs": int((obs.Close <= 0).sum()),
    "volume_negatifs": int((obs.Volume < 0).sum()),
    "volume_non_entiers": int((obs.Volume % 1 != 0).sum()),
    "week_ends": int((obs.Date.dt.weekday >= 5).sum()),
    "volumes_nuls": int((obs.Volume == 0).sum()),
    "volumes_nuls_avec_volume_FCFA_nul": int(((obs.Volume == 0) & (obs.Volume_FCFA == 0)).sum()),
    "calendrier_officiel_etabli": "statut_seance_officiel" in calendrier.columns,
    "jours_ouvres_civils_fenetre": int(np.busday_count(DEBUT.date(), (FIN +
pd.Timedelta(days=1)).date())),
    "jours_ouvres_civils_avec_observation": int(obs.Date.dt.weekday.lt(5).sum()),
}
audit["part_jours_ouvres_civils_avec_observation"] = round(audit["jours_ouvres_civils_avec_observation"]
/ audit["jours_ouvres_civils_fenetre"], 4)
audit["note_couverture"] = "dénominateur = jours ouvrés civils, fériés non connus : ce n'est pas un taux
de couverture des séances"
for k, v in audit.items():
    print(f"{k:50s} {v}")

par_annee = obs.groupby(obs.Date.dt.year).size().rename("observations")
par_trim = obs.groupby(obs.Date.dt.to_period("Q")).size().rename("observations")
print("\nObservations par année :", par_annee.to_dict())
print("Observations par trimestre :"); print(par_trim.to_string())
pd.DataFrame({"annee": par_annee.index, "observations": par_annee.values}).to_csv(os.path.join(SORTIE,
"tables", "effectifs_annuels.csv"), index=False)
par_trim.to_frame().reset_index().rename(columns={"Date": "trimestre"}).assign(trimestre=lambda d:
d.trimestre.astype(str)).to_csv(
    os.path.join(SORTIE, "tables", "effectifs_trimestriels.csv"), index=False)
pd.DataFrame([audit]).T.reset_index().rename(columns={"index": "controle", 0:
"valeur"}).to_csv(os.path.join(SORTIE, "tables", "audit_qualite.csv"), index=False)
```

```
lignes_ANALYSE                                  721
date_min                                        2023-01-02
date_max                                        2026-10-14
dates_non_parsees                               0
doublons_de_date                                0
close_manquants                                 0
volume_manquants                                0
close_non_positifs                              0
volume_negatifs                                 0
volume_non_entiers                              0
week_ends                                       0
volumes_nuls                                    47
volumes_nuls_avec_volume_FCFA_nul               47
calendrier_officiel_etabli                      False
jours_ouvres_civils_fenetre                     989
jours_ouvres_civils_avec_observation            721
part_jours_ouvres_civils_avec_observation       0.729
note_couverture                                 dénominateur = jours ouvrés civils, fériés non connus
: ce n'est pas un taux de couverture des séances
```

```
Observations par année : {2023: 177, 2024: 103, 2025: 247, 2026: 194}
Observations par trimestre :
Date
2023Q1    43
2023Q2    39
2023Q3    49
2023Q4    46
2024Q1    28
2024Q3    12
2024Q4    63
2025Q1    60
2025Q2    59
2025Q3    63
2025Q4    65
2026Q1    61
2026Q2    59
2026Q3    64
2026Q4    10
Freq: Q-DEC
```

C.2 Séances manquantes, suspensions et exclusions

Chaque observation exclue figure avec sa date, ses valeurs, sa raison et sa référence. Les jours ouvrés sans observation sont listés sans être comblés.

```
exclus = brut_idx[brut_idx.statut_analyse.str.startswith("EXCLUE")].copy()
exclus_tab = pd.DataFrame({
    "date": exclus.Date.dt.date.astype(str),
    "close": exclus.Close,
    "volume": exclus.Volume,
    "volume_FCFA": exclus.Volume_FCFA,
    "raison": exclus.statut_analyse.str.replace("EXCLUE : ", "", regex=False),
    "reference": "BRUT/SOLIBRA_SLBC_2023_2026_BRUT.csv ligne " + exclus.ligne_source.astype(str),
})
retires = journal_src[journal_src.operation == "exclusion (amont)"].copy()
retires_tab = pd.DataFrame({"date": retires.Date, "close": "", "volume": "", "volume_FCFA": "",
                            "raison": retires.justification, "reference": retires.reference})
exclusions = pd.concat([exclus_tab, retires_tab],
ignore_index=True).sort_values("date").reset_index(drop=True)
exclusions.to_csv(os.path.join(SORTIE, "tables", "exclusions_detaillees.csv"), index=False)
print(f"Exclusions détaillées : {len(exclus_tab)} observations de la fenêtre + {len(retires_tab)}
retraits amont par le dépôt")
display(exclus_tab) if "display" in globals() else print(exclus_tab.to_string(index=False))

jours_ouvres = pd.bdate_range(DEBUT, FIN)
absents = sorted(set(jours_ouvres) - set(obs.Date))
absents_tab = pd.DataFrame({"date": [d.date().isoformat() for d in absents],
                            "statut": "absence de ligne dans la source ; férié ou absence de cotation non
établi"})
absents_tab.to_csv(os.path.join(SORTIE, "tables", "jours_ouvres_sans_observation.csv"), index=False)
print(f"Jours ouvrés civils sans observation retenue : {len(absents)} (dont zones de suspension : "
      f"{sum(SUSP_DEBUT <= d <= SUSP_FIN for d in absents)})")
```

```
Exclusions détaillées : 4 observations de la fenêtre + 57 retraits amont par le dépôt
      date   close volume  volume_FCFA
raison                                    reference
2024-06-03  8325.0                0.0                                                cotation suspendue
(avis BRVM) BRUT/SOLIBRA_SLBC_2023_2026_BRUT.csv ligne 1882
2025-04-18 14195.0  231.0    3279045.0                                    contradiction non résolue
(vendredi saint) BRUT/SOLIBRA_SLBC_2023_2026_BRUT.csv ligne 2031
2026-04-03 38900.0   37.0    1439300.0                                    contradiction non résolue
(vendredi saint) BRUT/SOLIBRA_SLBC_2023_2026_BRUT.csv ligne 2269
2026-10-15 38250.0   44.0    1683000.0 contradiction non résolue (clôture provisoire contredite par le
```

```
snapshot BRVM) BRUT/SOLIBRA_SLBC_2023_2026_BRUT.csv ligne 2401
Jours ouvrés civils sans observation retenue : 268 (dont zones de suspension : 116)
```

D. Événements et préparation des séries

Les événements sur titre sont tirés des documents fournis : suspension, levée de suspension, fractionnement et dividendes.

```
ev = evenements.copy()
ev = ev[ev.categorie.isin(["trading_suspension", "trading_resumed", "theoretical_value_notice",
"dividend_ex_date", "dividend_payment"])]
print(ev[["date_evenement", "categorie", "titre"]].to_string(index=False))
print("\nFractionnement : métadonnées =", meta.get("date_effet_fractionnement"), "| source :",
meta.get("date_effet_source"))
print("Cours publiés ajustés du fractionnement :", meta.get("close_ajuste_fractionnement"))
```

```
date_evenement               categorie
titre
    2024-04-02       trading_suspension                                    SOLIBRA CI : Suspension du
titre
    2024-08-14         dividend_payment          SOLIBRA COTE D'IVOIRE : Dividende exercice 2023 -
Paiement
    2024-09-11          trading_resumed          SOLIBRA CÔTE D'IVOIRE : Levée de la suspension de
cotation
    2024-09-12         dividend_ex_date SOLIBRA COTE D'IVOIRE : Dividende exercice 2023 - Date ex-
dividende
    2024-09-27 theoretical_value_notice            SOLIBRA CI : Fractionnement d'actions & Valeur
théorique
    2025-07-25         dividend_ex_date               SOLIBRA : Dividende exercice 2024 - Date ex-
dividende
    2025-07-29         dividend_payment                        SOLIBRA : Dividende exercice 2024 -
Paiement
    2026-07-29         dividend_ex_date               SOLIBRA : Dividende exercice 2025 - Date ex-
dividende
    2026-07-30         dividend_payment                        SOLIBRA : Dividende exercice 2025 -
Paiement

Fractionnement : métadonnées = 2024-09-30 | source : presse (CGF Bourse) ; avis BRVM n°242 non lu
Cours publiés ajustés du fractionnement : NON
```

D.2 Statut de la base des cours

Les cours publiés ne sont pas ajustés du fractionnement de 2024. Tout calcul qui traverse la date du fractionnement est donc bloqué.

```
def bloque_par_split(p, c):
    return (p < SPLIT_DATE) and (c >= SPLIT_DATE)

def bloque_par_suspension(p, c):
    return (p <= SUSP_FIN) and (c >= SUSP_DEBUT)

def jours_ouvres_entre(p, c):
    return int(np.busday_count((p + pd.Timedelta(days=1)).date(), c.date()))

paires = []
for i in range(1, len(obs)):
    p, c = obs.loc[i - 1], obs.loc[i]
    gap = jours_ouvres_entre(p.Date, c.Date)
    if bloque_par_split(p.Date, c.Date):
        classe = "bloqué : traverse le fractionnement (base indéterminée)"
    elif bloque_par_suspension(p.Date, c.Date):
        classe = "bloqué : traverse la suspension"
    elif gap == 0:
        classe = "ouvrés consécutifs (continuité non démontrée)"
    else:
        classe = "jours ouvrés absents entre observations"
    paires.append({
        "date_prec": p.Date, "date": c.Date, "close_prec": p.Close, "close": c.Close,
        "volume_prec": p.Volume, "volume": c.Volume,
        "jours_ouvres_absents": gap, "classe": classe,
        "volume_nul_implique": (p.Volume == 0) or (c.Volume == 0),
    })
paires = pd.DataFrame(paires)
paires["r_simple"] = paires.close / paires.close_prec - 1
paires["r_log"] = np.log(paires.close / paires.close_prec)
paires["segment"] = np.select(
    [paires.date.between(*SEGMENTS["S1 pré-suspension"]), paires.date.between(*SEGMENTS["S2 reprise pré-
fractionnement"]),
     paires.date.between(*SEGMENTS["S3 post-fractionnement"])],
    list(SEGMENTS.keys()), default="hors segment")

print(paires.classe.value_counts().to_string())
print("\nRendements bloqués :", int(paires.classe.str.startswith("bloqué").sum()))
paires.to_csv(os.path.join(SORTIE, "tables", "paires_successives_toutes.csv"), index=False)

R_Q = paires[paires.classe.str.startswith("ouvrés consécutifs")].copy()
R_GAP = paires[paires.classe.str.startswith("jours ouvrés absents")].copy()
R_TOUS = paires[~paires.classe.str.startswith("bloqué")].copy()
R_Q_SANS_VOL0 = R_Q[~R_Q.volume_nul_implique].copy()
R_Q_SANS_VOL0_SEG = R_Q_SANS_VOL0
print(f"\nEffectifs : quotidiens admissibles {len(R_Q)} | avec lacunes {len(R_GAP)} | tous non bloqués
{len(R_TOUS)} | "
      f"quotidiens sans séance à volume nul {len(R_Q_SANS_VOL0)}")
print("Continuité quotidienne démontrée : NON (aucun calendrier officiel)")
```

```
classe
ouvrés consécutifs (continuité non démontrée)              637
jours ouvrés absents entre observations                     81
bloqué : traverse la suspension                              1
bloqué : traverse le fractionnement (base indéterminée)      1

Rendements bloqués : 2
```

```
Effectifs : quotidiens admissibles 637 | avec lacunes 81 | tous non bloqués 718 | quotidiens sans séance
à volume nul 573
Continuité quotidienne démontrée : NON (aucun calendrier officiel)
```

D.3 Séries conservées séparément

Trois séries sont conservées séparément : la série source, l'absence de série ajustée et l'échantillon retenu pour chaque analyse.

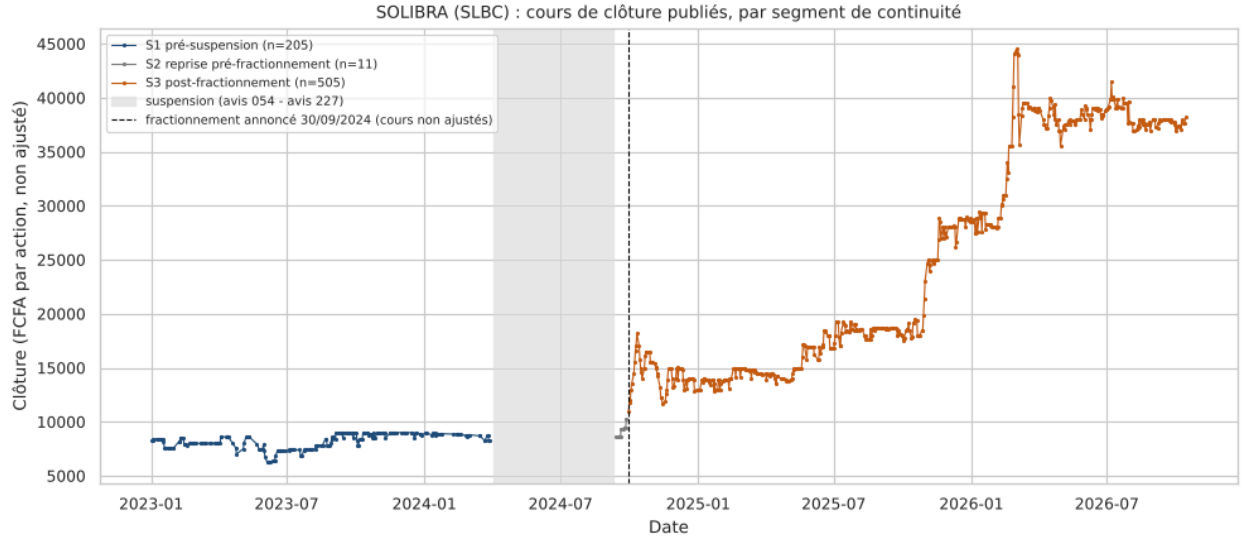
```
obs.to_csv(os.path.join(SORTIE, "tables", "serie_source_retenue.csv"), index=False)
print("Série source conservée :", len(obs), "observations ; série ajustée : aucune")
```

```
Série source conservée : 721 observations ; série ajustée : aucune
```

E. Cours, performance et rendements

Cette partie décrit l'évolution des cours, la performance, les baisses maximales et la distribution des rendements.

```
fig, ax = plt.subplots(figsize=(10, 4.5))
couleurs = {"S1 pré-suspension": "#1f4e79", "S2 reprise pré-fractionnement": "#7f7f7f", "S3 post-
fractionnement": "#c55a11"}
for nom, (a, b) in SEGMENTS.items():
    s = obs[(obs.Date >= a) & (obs.Date <= b)]
    ax.plot(s.Date, s.Close, marker=".", ms=2.5, lw=0.8, color=couleurs[nom], label=f"{nom} (n=
{len(s)})")
ax.axvspan(SUSP_DEBUT, SUSP_FIN, color="#dddddd", alpha=0.6, label="suspension (avis 054 - avis 227)")
ax.axvline(SPLIT_DATE, color="black", ls="--", lw=0.8, label="fractionnement annoncé 30/09/2024 (cours
non ajustés)")
ax.set_title("SOLIBRA (SLBC) : cours de clôture publiés, par segment de continuité")
ax.set_ylabel("Clôture (FCFA par action, non ajusté)")
ax.set_xlabel("Date")
ax.legend(fontsize=7, loc="upper left")
fig.text(0.01, -0.02, "Note : pas de liaison entre segments ; la série n'est ni ajustée ni validée.
Source : dépôt harzat/brvm-data-public.", fontsize=7)
fig.tight_layout()
fig.savefig(os.path.join(SORTIE, "figures", "01_cours_par_segment.png"), dpi=dpi_png,
bbox_inches="tight")
fig.savefig(os.path.join(SORTIE, "figures", "01_cours_par_segment.pdf"), bbox_inches="tight")
plt.show(); plt.close(fig)
```

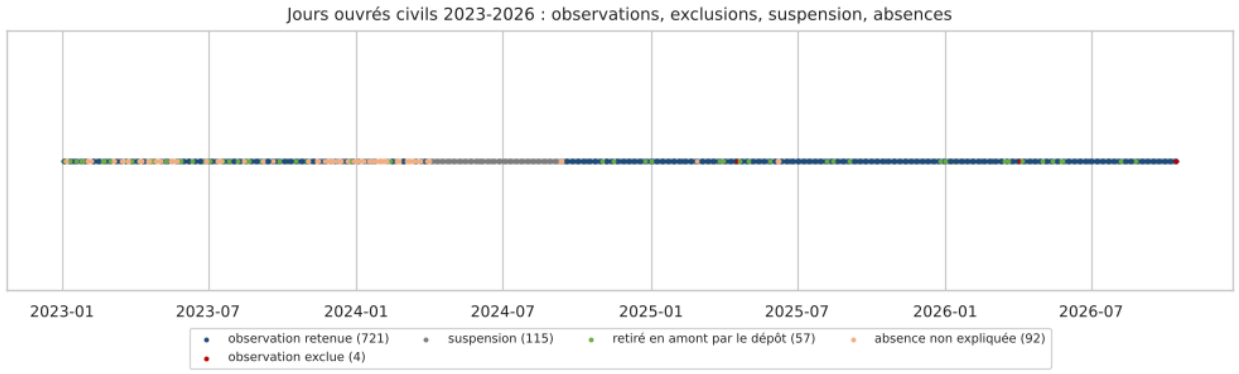


Note : pas de liaison entre segments ; la série n'est ni ajustée ni validée. Source : dépôt harzat/brvm-data-public.

E.2 Couverture et périodes absentes

Chaque jour ouvré civil est classé : observation retenue, observation exclue, suspension, retrait par la source ou absence non expliquée.

```
statut_jour = pd.Series("absence non expliquée", index=pd.bdate_range(DEBUT, FIN))
statut_jour[statut_jour.index.isin(obs.Date)] = "observation retenue"
statut_jour[(statut_jour.index >= SUSP_DEBUT) & (statut_jour.index <= SUSP_FIN)] = "suspension"
statut_jour[statut_jour.index.isin(exclus.Date)] = "observation exclue"
statut_jour[statut_jour.index.isin(pd.to_datetime(retires.Date))] = "retiré en amont par le dépôt"
ordre = {"observation retenue": 0, "observation exclue": 1, "suspension": 2, "retiré en amont par le
dépôt": 3, "absence non expliquée": 4}
couv = statut_jour.to_frame("statut")
couv["niveau"] = couv.statut.map(ordre)
couv["annee"] = couv.index.year
couv["jour_annee"] = couv.index.dayofyear
fig, ax = plt.subplots(figsize=(10, 3.2))
palette = {"observation retenue": "#1f4e79", "observation exclue": "#c00000", "suspension": "#7f7f7f",
           "retiré en amont par le dépôt": "#70ad47", "absence non expliquée": "#f4b183"}
for st, col in palette.items():
    s = couv[couv.statut == st]
    ax.scatter(s.index, np.zeros(len(s)), s=4, color=col, label=f"{st} ({len(s)})")
ax.set_yticks([])
ax.set_title("Jours ouvrés civils 2023-2026 : observations, exclusions, suspension, absences")
ax.legend(ncol=4, fontsize=7, loc="upper center", bbox_to_anchor=(0.5, -0.12))
fig.text(0.01, -0.2, "Note : 'absence non expliquée' = jour ouvré civil sans ligne ; férié ou absence de
cotation non établis.", fontsize=7)
fig.tight_layout()
fig.savefig(os.path.join(SORTIE, "figures", "02_couverture.png"), dpi=dpi_png, bbox_inches="tight")
fig.savefig(os.path.join(SORTIE, "figures", "02_couverture.pdf"), bbox_inches="tight")
plt.show(); plt.close(fig)
couv_resume = couv.statut.value_counts().rename("jours_ouvres_civils").to_frame()
couv_resume.to_csv(os.path.join(SORTIE, "tables", "couverture_resume.csv"))
print(couv_resume)
```



Note : 'absence non expliquée' = jour ouvré civil sans ligne ; férié ou absence de cotation non établis.

```
                           jours_ouvres_civils
statut
observation retenue                        721
suspension                                 115
absence non expliquée                       92
retiré en amont par le dépôt                57
observation exclue                           4
```

E.3 Performance des cours

La performance est la variation du cours entre le premier et le dernier point observés dans chaque segment. Elle ne tient pas compte des dividendes.

```
def perf_segment(nom, a, b):
    s = obs[(obs.Date >= a) & (obs.Date <= b)]
    if len(s) < 2:
        return {"segment": nom, "debut": None, "fin": None, "n": len(s), "performance_cours": np.nan}
    return {"segment": nom, "debut": s.Date.iloc[0].date().isoformat(), "fin":
s.Date.iloc[-1].date().isoformat(),
            "n": len(s), "close_debut": s.Close.iloc[0], "close_fin": s.Close.iloc[-1],
            "performance_cours": s.Close.iloc[-1] / s.Close.iloc[0] - 1}

perf_seg = pd.DataFrame([perf_segment(n, a, b) for n, (a, b) in SEGMENTS.items()])
perf_an = pd.DataFrame([
    perf_segment("2023 (année complète présente)", pd.Timestamp("2023-01-01"), pd.Timestamp("2023-12-
31")),
    perf_segment("2024 partie S1 (jan-mars)", pd.Timestamp("2024-01-01"), pd.Timestamp("2024-03-31")),
    perf_segment("2024 partie S3 (sept-déc)", pd.Timestamp("2024-09-30"), pd.Timestamp("2024-12-31")),
    perf_segment("2025 (complète)", pd.Timestamp("2025-01-01"), pd.Timestamp("2025-12-31")),
    perf_segment("2026 PARTIELLE (jusqu'au 14/10)", pd.Timestamp("2026-01-01"), pd.Timestamp("2026-12-
31")),
])
perf_seg.to_csv(os.path.join(SORTIE, "tables", "performance_segments.csv"), index=False)
perf_an.to_csv(os.path.join(SORTIE, "tables", "performance_par_annee.csv"), index=False)
print(perf_seg.to_string(index=False)); print()
print(perf_an.to_string(index=False))
print("\nPerformance sur toute la période : NON CALCULÉE (traverse le fractionnement, base
indéterminée).")
```

```
                    segment      debut        fin   n  close_debut  close_fin  performance_cours
          S1 pré-suspension 2023-01-02 2024-03-28 205         8325       8325           0.000000
S2 reprise pré-fractionnement 2024-09-13 2024-09-27  11         8659      10213           0.179466
       S3 post-fractionnement 2024-09-30 2026-10-14 505        10975      38200           2.480638

                        segment      debut        fin   n  close_debut  close_fin  performance_cours
 2023 (année complète présente) 2023-01-02 2023-12-28 177         8325       8700           0.045045
      2024 partie S1 (jan-mars) 2024-01-02 2024-03-28  28         9000       8325          -0.075000
      2024 partie S3 (sept-déc) 2024-09-30 2024-12-31  64        10975      12950           0.179954
                2025 (complète) 2025-01-02 2025-12-31 247        13000      28900           1.223077
2026 PARTIELLE (jusqu'au 14/10) 2026-01-02 2026-10-14 194        28500      38200           0.340351

Performance sur toute la période : NON CALCULÉE (traverse le fractionnement, base indéterminée).
```

E.4 Drawdown et récupération

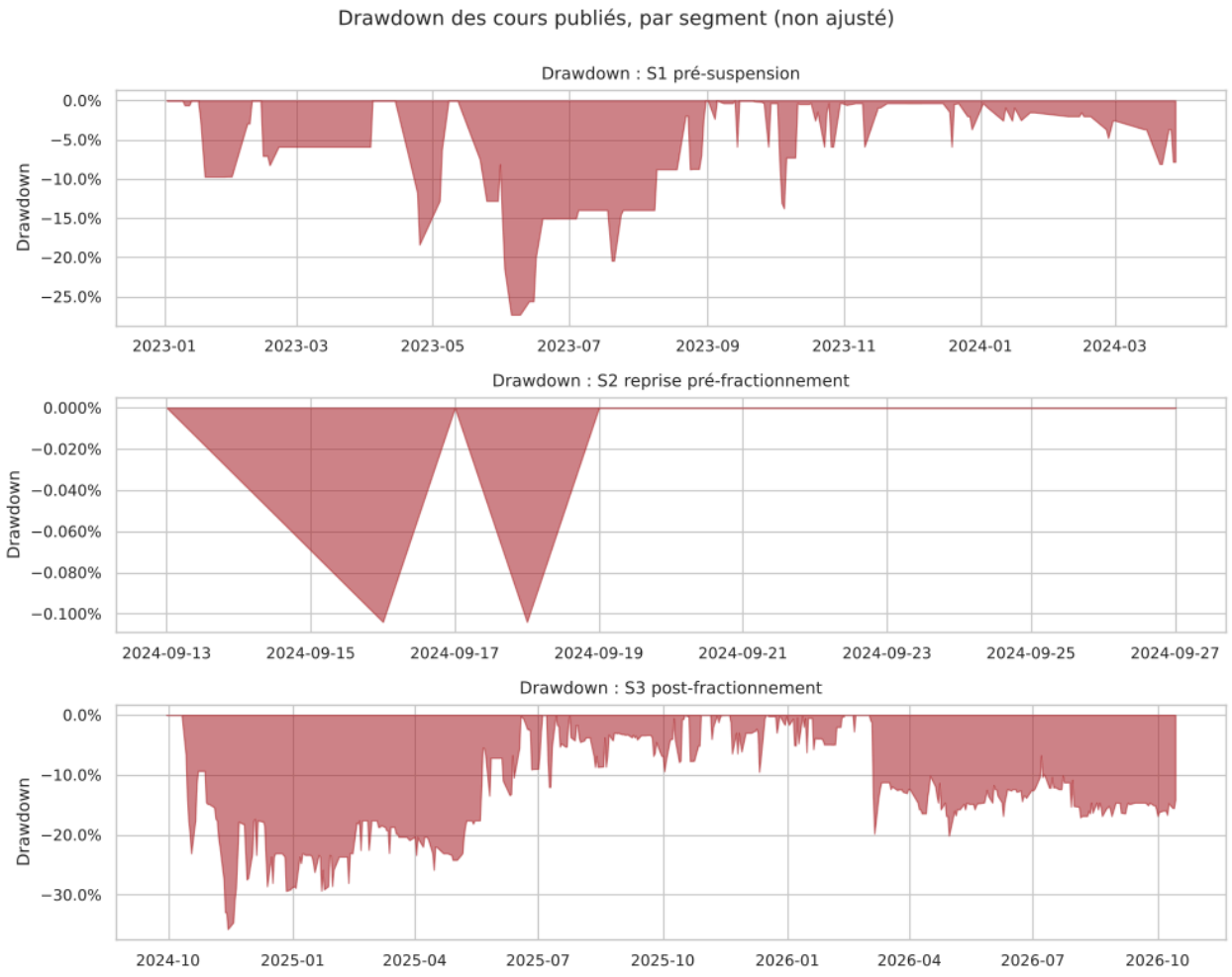
Le drawdown mesure la baisse depuis le plus haut précédent. Le tableau donne les dates du sommet et du creux, et la date de récupération lorsqu'elle est observée.

```
def drawdown_segment(nom, a, b):
    s = obs[(obs.Date >= a) & (obs.Date <= b)].reset_index(drop=True)
    if len(s) < 2:
        return None, None
    pic = s.Close.cummax()
    dd = s.Close / pic - 1
    i_creux = int(dd.idxmin())
    i_sommet = int(s.Close.iloc[: i_creux + 1].idxmax())
    apres = s.iloc[i_creux:]
    rec = apres[apres.Close >= s.Close.iloc[i_sommet]]
    if len(rec):
        j = rec.index[0]
        recup = {"date_recuperation": s.Date.iloc[j].date().isoformat(),
                 "jours_ouvres_recuperation_observe": int(np.busday_count(s.Date.iloc[i_creux].date(),
s.Date.iloc[j].date())),
                 "n_observations_recuperation": int(j - i_creux)}
    else:
        recup = {"date_recuperation": "non observée dans le segment",
"jours_ouvres_recuperation_observe": np.nan,
                 "n_observations_recuperation": np.nan}
    return {"segment": nom, "max_drawdown": dd.iloc[i_creux], "date_sommet":
s.Date.iloc[i_sommet].date().isoformat(),
            "cours_sommet": s.Close.iloc[i_sommet], "date_creux":
s.Date.iloc[i_creux].date().isoformat(),
            "cours_creux": s.Close.iloc[i_creux], **recup}, pd.DataFrame({"Date": s.Date, "dd":
dd.values})

dd_rows, dd_series = [], {}
for nom, (a, b) in SEGMENTS.items():
    r, dser = drawdown_segment(nom, a, b)
    if r:
        dd_rows.append(r); dd_series[nom] = dser
dd_tab = pd.DataFrame(dd_rows)
dd_tab.to_csv(os.path.join(SORTIE, "tables", "drawdown_par_segment.csv"), index=False)
print(dd_tab.to_string(index=False))

fig, axes = plt.subplots(len(dd_series), 1, figsize=(10, 2.6 * len(dd_series)), sharex=False)
for ax, (nom, dser) in zip(np.atleast_1d(axes), dd_series.items()):
    ax.fill_between(dser.Date, dser.dd.values, 0, color="#a50f15", alpha=0.5)
    ax.set_title(f"Drawdown : {nom}"); ax.set_ylabel("Drawdown")
    ax.yaxis.set_major_formatter(matplotlib.ticker.PercentFormatter(1.0))
fig.suptitle("Drawdown des cours publiés, par segment (non ajusté)", y=1.0)
fig.tight_layout()
fig.savefig(os.path.join(SORTIE, "figures", "03_drawdown.png"), dpi=dpi_png, bbox_inches="tight")
fig.savefig(os.path.join(SORTIE, "figures", "03_drawdown.pdf"), bbox_inches="tight")
plt.show(); plt.close(fig)
```

```
                    segment  max_drawdown date_sommet  cours_sommet date_creux  cours_creux
date_recuperation  jours_ouvres_recuperation_observe  n_observations_recuperation
           S1 pré-suspension     -0.272907  2023-04-04          8600 2023-06-05         6253
2023-08-31                                63                          47
S2 reprise pré-fractionnement     -0.001039  2024-09-13          8659 2024-09-16         8650
2024-09-17                                 1                           1
       S3 post-fractionnement     -0.357143  2024-10-11         18200 2024-11-14        11700
2025-06-18                               154                         142
```



E.5 Statistiques des rendements

Les statistiques portent sur les rendements admissibles, c'est-à-dire ceux entre deux séances consécutives sans lacune et hors suspension ou fractionnement.

```
def stats_rendements(x):
    x = pd.Series(x).dropna()
    n = len(x)
    if n < 3:
        return {"n": n}
    q = x.quantile([0.01, 0.05, 0.25, 0.5, 0.75, 0.95, 0.99])
    return {"n": n, "moyenne": x.mean(), "mediane": x.median(), "ecart_type": x.std(ddof=1),
            "minimum": x.min(), "maximum": x.max(),
            "q01": q.loc[0.01], "q05": q.loc[0.05], "q25": q.loc[0.25], "q75": q.loc[0.75], "q95":
q.loc[0.95], "q99": q.loc[0.99],
            "asymetrie_ajustee": stats.skew(x, bias=False), "kurtosis_exces_ajustee": stats.kurtosis(x,
fisher=True, bias=False),
            "proportion_nuls": float((x == 0).mean()), "n_nuls": int((x == 0).sum())}

stat_tab = pd.DataFrame({
    "R_Q simple": stats_rendements(R_Q.r_simple),
    "R_Q log": stats_rendements(R_Q.r_log),
    "R_GAP simple (lacunes, séparé)": stats_rendements(R_GAP.r_simple),
}).T
stat_tab.to_csv(os.path.join(SORTIE, "tables", "statistiques_rendements.csv"))
pd.set_option("display.float_format", lambda v: f"{v:,.5f}")
print(stat_tab.to_string())
pd.set_option("display.float_format", None)
```

```
                                    n  moyenne  mediane  ecart_type  minimum  maximum      q01
q05      q25     q75     q95     q99  asymetrie_ajustee  kurtosis_exces_ajustee  proportion_nuls
n_nuls
R_Q simple                      637.00000  0.00300  0.00000     0.03150 -0.12400  0.07507 -0.07492
-0.05708 -0.00463 0.00976 0.07344 0.07500            0.16766                 1.34841          0.27316
174.00000
R_Q log                         637.00000  0.00250  0.00000     0.03138 -0.13239  0.07238 -0.07788
-0.05877 -0.00464 0.00971 0.07087 0.07232            0.00802                 1.47033          0.27316
174.00000
R_GAP simple (lacunes, séparé)  81.00000 -0.00184  0.00000     0.02878 -0.07500  0.07486 -0.07500
-0.05720 -0.00677 0.00054 0.05294 0.07483           -0.03296                 2.18280          0.35802
29.00000
```

F. Volatilité, drawdown et risque historique

Cette partie porte sur la volatilité, le risque historique et les volumes publiés.

```
vol_obs = {
    "écart-type quotidien admissible (log)": R_Q.r_log.std(ddof=1),
    "écart-type quotidien admissible (simple)": R_Q.r_simple.std(ddof=1),
    "effectif": len(R_Q),
    "annualisation": "non effectuée : pas de calendrier officiel ni de nombre de séances documenté",
}
vol_seg = pd.DataFrame([{"segment": n, "n": int(((R_Q.segment == n)).sum()),
                         "ecart_type_log": R_Q.loc[R_Q.segment == n, "r_log"].std(ddof=1)} for n in
SEGMENTS])
vol_seg.to_csv(os.path.join(SORTIE, "tables", "volatilite_par_segment.csv"), index=False)
print({k: (round(v, 6) if isinstance(v, float) else v) for k, v in vol_obs.items()})
print(vol_seg.to_string(index=False))
```

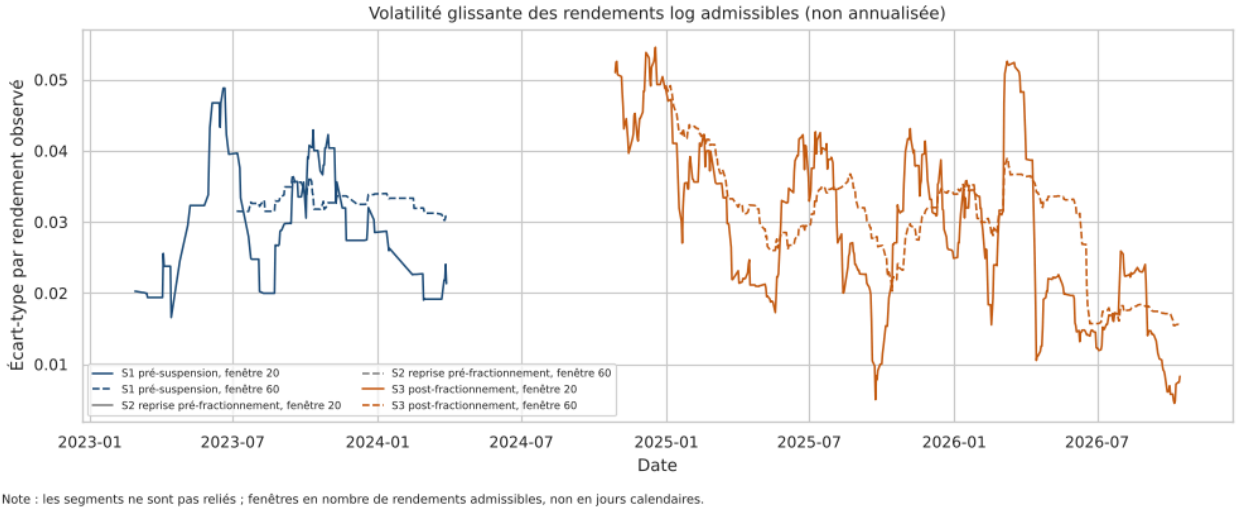
```
{'écart-type quotidien admissible (log)': np.float64(0.031384), 'écart-type quotidien admissible
(simple)': np.float64(0.031503), 'effectif': 637, 'annualisation': 'non effectuée : pas de calendrier
officiel ni de nombre de séances documenté'}
                     segment   n  ecart_type_log
           S1 pré-suspension 148        0.030521
S2 reprise pré-fractionnement  10        0.030107
       S3 post-fractionnement 479        0.031667
```

F.2 Volatilité glissante

La volatilité glissante est calculée sur 20 et 60 rendements, à l'intérieur de chaque segment.

```
roul = []
for nom in SEGMENTS:
    s = R_Q[R_Q.segment == nom].sort_values("date").copy()
    for w in FENETRES_VOL:
        s[f"vol_glissante_{w}"] = s.r_log.rolling(w, min_periods=w).std(ddof=1)
    roul.append(s)
roul = pd.concat(roul)
roul.to_csv(os.path.join(SORTIE, "tables", "volatilite_glissante.csv"), index=False)

fig, ax = plt.subplots(figsize=(10, 4))
for nom, col in couleurs.items():
    s = roul[roul.segment == nom]
    for w, ls in zip(FENETRES_VOL, ["-", "--"]):
        ax.plot(s.date, s[f"vol_glissante_{w}"], ls=ls, color=col, lw=1.1, label=f"{nom}, fenêtre {w}")
ax.set_title("Volatilité glissante des rendements log admissibles (non annualisée)")
ax.set_ylabel("Écart-type par rendement observé"); ax.set_xlabel("Date")
ax.legend(fontsize=6, ncol=2)
fig.text(0.01, -0.02, "Note : les segments ne sont pas reliés ; fenêtres en nombre de rendements
admissibles, non en jours calendaires.", fontsize=7)
fig.tight_layout()
fig.savefig(os.path.join(SORTIE, "figures", "04_volatilite_glissante.png"), dpi=dpi_png,
bbox_inches="tight")
fig.savefig(os.path.join(SORTIE, "figures", "04_volatilite_glissante.pdf"), bbox_inches="tight")
plt.show(); plt.close(fig)
```



Note : les segments ne sont pas reliés ; fenêtres en nombre de rendements admissibles, non en jours calendaires.

F.3 VaR et Expected Shortfall historiques

Les pertes historiques à 95 % et 99 % sont calculées à un jour. Les intervalles de confiance utilisent un bootstrap par blocs, qui conserve la dépendance entre observations voisines.

```
def var_es(x, niveau, bloc=BLOC, B=B_BOOT, graine=GRAINE):
    L = -np.asarray(x, dtype=float)
    n = len(L)
    if n < 2:
        return {"niveau": niveau, "n": n}
    v = np.quantile(L, niveau, method="inverted_cdf")
    queue = L[L >= v]
    es = queue.mean()
    rng = np.random.default_rng(graine)
    nb = int(np.ceil(n / bloc))
    starts_max = n - bloc
    vs, ess = [], []
    for _ in range(B):
        idx = np.concatenate([np.arange(s, s + bloc) for s in rng.integers(0, starts_max + 1, nb)])[:n]
        Lb = L[idx]
        vb = np.quantile(Lb, niveau, method="inverted_cdf")
        vs.append(vb)
        ess.append(Lb[Lb >= vb].mean())
    return {"niveau": niveau, "n": n, "obs_dans_queue": int(len(queue)),
            "VaR": v, "VaR_IC95_bas": np.quantile(vs, 0.025), "VaR_IC95_haut": np.quantile(vs, 0.975),
            "ES": es, "ES_IC95_bas": np.quantile(ess, 0.025), "ES_IC95_haut": np.quantile(ess, 0.975),
            "avertissement": "queue < MIN_QUEUE : estimation instable" if len(queue) < MIN_QUEUE else ""}

ve_rows = []
for nom, x in [("R_Q toutes périodes", R_Q.r_simple), ("R_Q S1", R_Q[R_Q.segment == "S1 pré-
suspension"].r_simple),
               ("R_Q S3", R_Q[R_Q.segment == "S3 post-fractionnement"].r_simple)]:
    for niv in NIVEAUX_VAR:
        ve_rows.append({"ensemble": nom, **var_es(x, niv)})
ve_tab = pd.DataFrame(ve_rows)
ve_tab.to_csv(os.path.join(SORTIE, "tables", "var_es_historiques.csv"), index=False)
pd.set_option("display.float_format", lambda v: f"{v:,.4f}")
print(ve_tab.to_string(index=False))
pd.set_option("display.float_format", None)
```

```
           ensemble  niveau   n  obs_dans_queue    VaR  VaR_IC95_bas  VaR_IC95_haut     ES  ES_IC95_bas
ES_IC95_haut                            avertissement
R_Q toutes périodes  0.9500 637              32 0.0572        0.0456         0.0667 0.0700       0.0625
0.0764
R_Q toutes périodes  0.9900 637               7 0.0749        0.0731         0.0750 0.0820       0.0746
0.0960 queue < MIN_QUEUE : estimation instable
             R_Q S1  0.9500 148               8 0.0587        0.0238         0.0750 0.0717       0.0509
0.0750 queue < MIN_QUEUE : estimation instable
             R_Q S1  0.9900 148               2 0.0750        0.0695         0.0750 0.0750       0.0706
0.0750 queue < MIN_QUEUE : estimation instable
             R_Q S3  0.9500 479              24 0.0572        0.0441         0.0659 0.0695       0.0601
0.0776 queue < MIN_QUEUE : estimation instable
             R_Q S3  0.9900 479               5 0.0748        0.0704         0.0750 0.0847       0.0736
0.1044 queue < MIN_QUEUE : estimation instable
```

F.4 Volumes publiés

Les volumes publiés sont comparés uniquement à l'intérieur d'un même régime d'unités d'action, avant ou après le fractionnement.

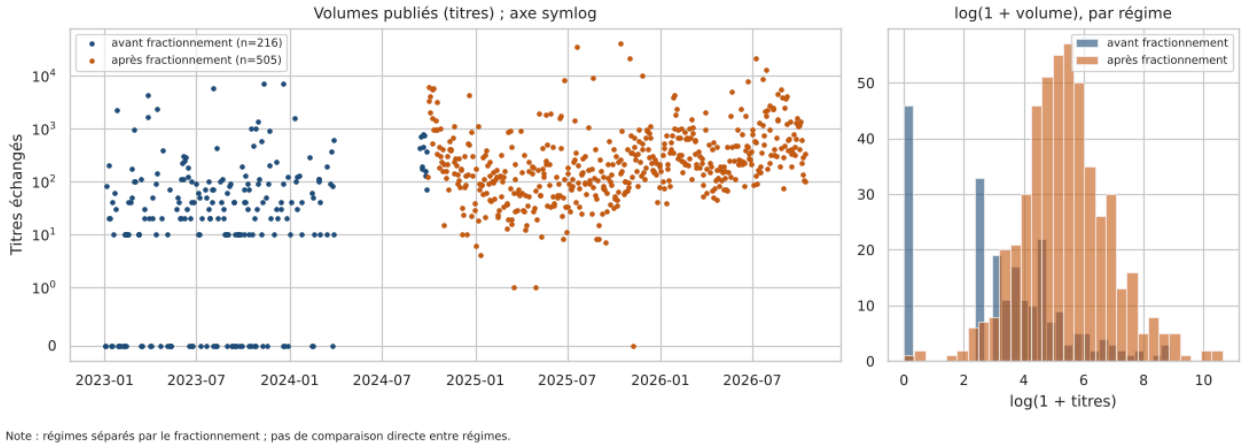
```
obs["regime_unites"] = np.where(obs.Date < SPLIT_DATE, "avant fractionnement", "après fractionnement")
vol_reg = obs.groupby("regime_unites").agg(
    n=("Volume", "size"), mediane=("Volume", "median"), moyenne=("Volume", "mean"),
    q10=("Volume", lambda s: s.quantile(0.10)), q90=("Volume", lambda s: s.quantile(0.90)),
    nuls=("Volume", lambda s: int((s == 0).sum())), montant_publie_mediane=("Volume_FCFA", "median"))
vol_reg.to_csv(os.path.join(SORTIE, "tables", "volumes_par_regime.csv"))
print(vol_reg.to_string())

out = []
for reg, s in obs.groupby("regime_unites"):
    lv = np.log1p(s.Volume)
    q1, q3 = lv.quantile(0.25), lv.quantile(0.75)
    iqr = q3 - q1
    m = (lv < q1 - 1.5 * iqr) | (lv > q3 + 1.5 * iqr)
    out.append(s[m].assign(regime=reg))
vol_out = pd.concat(out)[["Date", "Close", "Volume", "Volume_FCFA", "regime_unites"]]
vol_out.to_csv(os.path.join(SORTIE, "tables", "volumes_atypiques_tukey.csv"), index=False)
print(f"\nValeurs atypiques de volume (Tukey dans chaque régime, conservées) : {len(vol_out)}")

fig, axes = plt.subplots(1, 2, figsize=(11, 3.8), gridspec_kw={"width_ratios": [2.2, 1]})
for reg, col in [("avant fractionnement", "#1f4e79"), ("après fractionnement", "#c55a11")]:
    s = obs[obs.regime_unites == reg]
    axes[0].scatter(s.Date, s.Volume, s=6, color=col, label=f"{reg} (n={len(s)})")
    axes[1].hist(np.log1p(s.Volume), bins=30, alpha=0.6, color=col, label=reg)
axes[0].set_yscale("symlog", linthresh=1); axes[0].set_title("Volumes publiés (titres) ; axe symlog")
axes[0].set_ylabel("Titres échangés"); axes[0].legend(fontsize=7)
axes[1].set_title("log(1 + volume), par régime"); axes[1].set_xlabel("log(1 + titres)");
axes[1].legend(fontsize=7)
fig.text(0.01, -0.03, "Note : régimes séparés par le fractionnement ; pas de comparaison directe entre
régimes.", fontsize=7)
fig.tight_layout()
fig.savefig(os.path.join(SORTIE, "figures", "05_volumes.png"), dpi=dpi_png, bbox_inches="tight")
fig.savefig(os.path.join(SORTIE, "figures", "05_volumes.pdf"), bbox_inches="tight")
plt.show(); plt.close(fig)
```

```
                        n  mediane     moyenne   q10     q90  nuls  montant_publie_mediane
regime_unites
après fractionnement  505    206.0  827.401980  35.0  1394.0     1               4841010.0
avant fractionnement  216     30.0  245.324074   0.0   405.0    46                269205.0

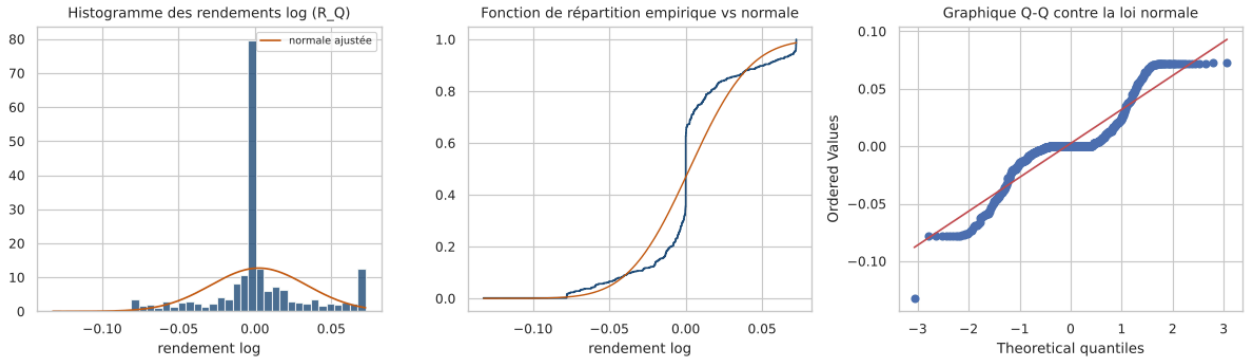
Valeurs atypiques de volume (Tukey dans chaque régime, conservées) : 18
```



G. Diagnostics statistiques et robustesse

Cette partie teste la normalité, la dépendance temporelle et la stationnarité, puis vérifie la sensibilité des résultats à l'échantillon.

```
x = R_Q.r_log.values
fig, axes = plt.subplots(1, 3, figsize=(12, 3.6))
axes[0].hist(x, bins=40, color="#1f4e79", alpha=0.8, density=True)
mu, sd = x.mean(), x.std(ddof=1)
grille = np.linspace(x.min(), x.max(), 200)
axes[0].plot(grille, stats.norm.pdf(grille, mu, sd), color="#c55a11", lw=1.2, label="normale ajustée")
axes[0].set_title("Histogramme des rendements log (R_Q)"); axes[0].set_xlabel("rendement log");
axes[0].legend(fontsize=7)
xs = np.sort(x); fdr = np.arange(1, len(xs) + 1) / len(xs)
axes[1].step(xs, fdr, where="post", color="#1f4e79"); axes[1].plot(grille, stats.norm.cdf(grille, mu,
sd), color="#c55a11", lw=1)
axes[1].set_title("Fonction de répartition empirique vs normale"); axes[1].set_xlabel("rendement log")
stats.probplot(x, dist="norm", plot=axes[2])
axes[2].set_title("Graphique Q-Q contre la loi normale")
fig.tight_layout()
fig.savefig(os.path.join(SORTIE, "figures", "06_distribution_qq.png"), dpi=dpi_png, bbox_inches="tight")
fig.savefig(os.path.join(SORTIE, "figures", "06_distribution_qq.pdf"), bbox_inches="tight")
plt.show(); plt.close(fig)
```



G.2 Autocorrélation, Ljung-Box, effet ARCH et stationnarité

Les tests d'autocorrélation, d'effet ARCH et de stationnarité sont appliqués séparément à chaque segment.

```
def tests_serie(nom, rend, prix):
    rend = np.asarray(rend, dtype=float)
    out = {"serie": nom, "n_rendements": len(rend)}
    if len(rend) < 30:
        out["note"] = "effectif < 30 : tests non interprétables"
        return out
    lb = acorr_ljungbox(rend, lags=LAGS_LJUNG, return_df=True)
    lb2 = acorr_ljungbox(rend ** 2, lags=LAGS_LJUNG, return_df=True)
    for k in LAGS_LJUNG:
        out[f"LjungBox_rend_p_lag{k}"] = float(lb.loc[k, "lb_pvalue"])
        out[f"LjungBox_carres_p_lag{k}"] = float(lb2.loc[k, "lb_pvalue"])
    arch = het_arch(rend - rend.mean(), nlags=LAGS_ARCH)
    out["ARCH_LM_stat"], out["ARCH_LM_p"] = float(arch[0]), float(arch[1])
    jb = stats.jarque_bera(rend)
    out["JarqueBera_stat"], out["JarqueBera_p"] = float(jb.statistic), float(jb.pvalue)
    if len(rend) <= 5000:
        sw = stats.shapiro(rend)
        out["ShapiroWilk_stat"], out["ShapiroWilk_p"] = float(sw.statistic), float(sw.pvalue)
    adf_rend = adfuller(rend, regression="c", autolag="AIC")
    out["ADF_rend_p"] = float(adf_rend[1]); out["ADF_rend_retards"] = int(adf_rend[2])
    kp = kpss(rend, regression="c", nlags="auto")
    out["KPSS_rend_p"] = float(kp[1]); out["KPSS_rend_retards"] = int(kp[2])
    if prix is not None and len(prix) >= 30:
        lp = np.log(np.asarray(prix, dtype=float))
        adf_p = adfuller(lp, regression="c", autolag="AIC")
        out["ADF_log_cours_p"] = float(adf_p[1]); out["ADF_log_cours_retards"] = int(adf_p[2])
        kp_p = kpss(lp, regression="c", nlags="auto")
        out["KPSS_log_cours_p"] = float(kp_p[1])
    return out

tests = []
for nom in ["S1 pré-suspension", "S3 post-fractionnement"]:
    s_r = R_Q[R_Q.segment == nom].sort_values("date")
    s_p = obs[(obs.Date >= SEGMENTS[nom][0]) & (obs.Date <= SEGMENTS[nom][1])].Close
    tests.append(tests_serie(nom, s_r.r_log, s_p))
tests_tab = pd.DataFrame(tests)
tests_tab.to_csv(os.path.join(SORTIE, "tables", "tests_diagnostics.csv"), index=False)
pd.set_option("display.float_format", lambda v: f"{v:,.4f}")
print(tests_tab.T.to_string())
pd.set_option("display.float_format", None)
```

```
                                    0                      1
serie                   S1 pré-suspension  S3 post-fractionnement
n_rendements                          148                    479
LjungBox_rend_p_lag5               0.4398                 0.0113
LjungBox_carres_p_lag5             0.2108                 0.0000
LjungBox_rend_p_lag10              0.4735                 0.0943
LjungBox_carres_p_lag10            0.0106                 0.0000
ARCH_LM_stat                       7.5549                52.4025
ARCH_LM_p                          0.1825                 0.0000
JarqueBera_stat                   19.2005                36.7738
JarqueBera_p                       0.0001                 0.0000
ShapiroWilk_stat                   0.7727                 0.9052
ShapiroWilk_p                      0.0000                 0.0000
ADF_rend_p                         0.0000                 0.0000
ADF_rend_retards                        1                      3
KPSS_rend_p                        0.1000                 0.1000
KPSS_rend_retards                       6                      2
ADF_log_cours_p                    0.0690                 0.6355
ADF_log_cours_retards                   1                      0
KPSS_log_cours_p                   0.0100                 0.0100
```

```
/tmp/ipykernel_826/2444552512.py:21: InterpolationWarning: The test statistic is outside of the range of
p-values available in the look-up table. The actual p-value is greater than the p-value returned.
  kp = kpss(rend, regression="c", nlags="auto")
/tmp/ipykernel_826/2444552512.py:27: InterpolationWarning: The test statistic is outside of the range of
p-values available in the look-up table. The actual p-value is smaller than the p-value returned.
```

```
  kp_p = kpss(lp, regression="c", nlags="auto")
/tmp/ipykernel_826/2444552512.py:21: InterpolationWarning: The test statistic is outside of the range of
p-values available in the look-up table. The actual p-value is greater than the p-value returned.
  kp = kpss(rend, regression="c", nlags="auto")
/tmp/ipykernel_826/2444552512.py:27: InterpolationWarning: The test statistic is outside of the range of
p-values available in the look-up table. The actual p-value is smaller than the p-value returned.
  kp_p = kpss(lp, regression="c", nlags="auto")
```

G.3 Intervalles de confiance avec dépendance temporelle

Les intervalles de confiance sont obtenus par bootstrap par blocs pour tenir compte de la dépendance temporelle.

```
def ic_bloc(x, fonction, bloc, B=B_BOOT, graine=GRAINE):
    x = np.asarray(x, dtype=float)
    n = len(x)
    rng = np.random.default_rng(graine)
    nb = int(np.ceil(n / bloc))
    vals = []
    for _ in range(B):
        idx = np.concatenate([np.arange(s, s + bloc) for s in rng.integers(0, n - bloc + 1, nb)])[:n]
        vals.append(fonction(x[idx]))
    vals = np.array(vals)
    return fonction(x), np.quantile(vals, 0.025), np.quantile(vals, 0.975)

def acf1(y):
    y = y - y.mean()
    return float((y[1:] * y[:-1]).sum() / (y * y).sum())

x3 = R_Q[R_Q.segment == "S3 post-fractionnement"].sort_values("date").r_log.values
boot_rows = []
for bloc in [BLOC, 2 * BLOC]:
    m, lo, hi = ic_bloc(x3, np.mean, bloc)
    boot_rows.append({"statistique": "moyenne des rendements log (S3)", "bloc": bloc, "estimation": m,
"IC95_bas": lo, "IC95_haut": hi, "n": len(x3)})
    a1, lo, hi = ic_bloc(x3, acf1, bloc)
    boot_rows.append({"statistique": "autocorrélation d'ordre 1 (S3)", "bloc": bloc, "estimation": a1,
"IC95_bas": lo, "IC95_haut": hi, "n": len(x3)})
boot_tab = pd.DataFrame(boot_rows)
boot_tab.to_csv(os.path.join(SORTIE, "tables", "ic_bootstrap_blocs.csv"), index=False)
pd.set_option("display.float_format", lambda v: f"{v:,.5f}")
print(boot_tab.to_string(index=False))
pd.set_option("display.float_format", None)
```

```
                  statistique  bloc  estimation  IC95_bas  IC95_haut   n
moyenne des rendements log (S3)     5     0.00240  -0.00061    0.00530 479
 autocorrélation d'ordre 1 (S3)     5     0.06564  -0.09034    0.17378 479
moyenne des rendements log (S3)    10     0.00240  -0.00041    0.00432 479
 autocorrélation d'ordre 1 (S3)    10     0.06564  -0.10954    0.17659 479
```

G.4 Robustesse

Les statistiques et les VaR sont recalculées sur plusieurs échantillons pour mesurer leur sensibilité.

```
def ligne_robustesse(nom, s):
    s = pd.Series(s).dropna()
    rec = {"echantillon": nom, "n": len(s), "moyenne": s.mean() if len(s) else np.nan,
           "ecart_type_log": np.log1p(s).std(ddof=1) if len(s) > 2 else np.nan}
    rec["ecart_type_simple"] = s.std(ddof=1) if len(s) > 2 else np.nan
    rec["asymetrie"] = stats.skew(s, bias=False) if len(s) > 2 else np.nan
    rec["kurtosis_exces"] = stats.kurtosis(s, fisher=True, bias=False) if len(s) > 3 else np.nan
    if len(s) >= 20:
        rec["VaR95"] = var_es(s, 0.95, B=500)["VaR"]
        rec["ES95"] = var_es(s, 0.95, B=500)["ES"]
    return rec

q_lo, q_hi = R_Q.r_simple.quantile([0.01, 0.99])
variantes = [
    ("quotidiens admissibles (principal)", R_Q.r_simple),
    ("tous non bloqués (avec lacunes)", R_TOUS.r_simple),
    ("lacunes seules", R_GAP.r_simple),
    ("quotidiens sans séance à volume nul", R_Q_SANS_VOL0.r_simple),
    ("quotidiens sans 1 % extrêmes (sensibilité)", R_Q.r_simple[(R_Q.r_simple >= q_lo) & (R_Q.r_simple <=
q_hi)]),
    ("S1 seul", R_Q[R_Q.segment == "S1 pré-suspension"].r_simple),
    ("S3 seul", R_Q[R_Q.segment == "S3 post-fractionnement"].r_simple),
    ("2025 seul", R_Q[R_Q.date.dt.year == 2025].r_simple),
    ("2026 jusqu'au 14/10 (partiel)", R_Q[R_Q.date.dt.year == 2026].r_simple),
]
rob = pd.DataFrame([ligne_robustesse(n, s) for n, s in variantes])
rob.to_csv(os.path.join(SORTIE, "tables", "robustesse.csv"), index=False)
pd.set_option("display.float_format", lambda v: f"{v:,.4f}")
print(rob.to_string(index=False))
pd.set_option("display.float_format", None)
```

```
                             echantillon   n  moyenne  ecart_type_log  ecart_type_simple  asymetrie
kurtosis_exces  VaR95   ES95
        quotidiens admissibles (principal) 637   0.0030          0.0314             0.0315     0.1677
1.3484 0.0572 0.0700
           tous non bloqués (avec lacunes) 718   0.0025          0.0311             0.0312     0.1578
1.4158 0.0572 0.0704
                           lacunes seules  81  -0.0018          0.0289             0.0288    -0.0330
2.1828 0.0572 0.0704
       quotidiens sans séance à volume nul 573   0.0025          0.0317             0.0317     0.1456
1.3467 0.0576 0.0704
quotidiens sans 1 % extrêmes (sensibilité) 628   0.0037          0.0299             0.0301     0.3666
1.2091 0.0506 0.0640
                                  S1 seul 148   0.0024          0.0305             0.0306     0.0795
1.7876 0.0587 0.0717
                                  S3 seul 479   0.0029          0.0317             0.0318     0.1730
1.2491 0.0572 0.0695
                                2025 seul 235   0.0036          0.0309             0.0311     0.3500
0.9730 0.0506 0.0648
             2026 jusqu'au 14/10 (partiel) 184   0.0016          0.0248             0.0247    -0.1444
5.1199 0.0395 0.0578
```

G.5 Lecture critique de la robustesse

Le tableau de robustesse est lu variante par variante. Aucune observation n'est retirée pour améliorer un résultat.

H. Comparaison au marché (BRVM Composite)

Le titre est comparé à l'indice BRVM Composite sur les dates communes.

```
ind = indice_brut.copy()
ind["Date"] = pd.to_datetime(ind.Date, format="%d/%m/%Y", errors="coerce")
ind["Close"] = pd.to_numeric(ind["Clôture"].str.replace(",", ".", regex=False), errors="coerce")
ind = ind[(ind.Date >= DEBUT) & (ind.Date <= FIN)].sort_values("Date").reset_index(drop=True)
print("Indice sur la fenêtre :", len(ind), "lignes ;", ind.Date.min().date(), "→", ind.Date.max().date())
print("Doublons :", int(ind.Date.duplicated().sum()), "| clôtures manquantes :",
int(ind.Close.isna().sum()))
copies_ind = int(((ind.Close == ind.Close.shift()) & (ind.Date.diff().dt.days <= 4)).sum())
print("Clôtures identiques à la séance précédente (indice) :", copies_ind,
      "— à noter : la clôture du 15/10/2026 répète celle du 14/10/2026")
ind["rv"] = ind.Close.pct_change()
ind_paires = []
for i in range(1, len(ind)):
    p, c = ind.loc[i - 1], ind.loc[i]
    ind_paires.append({"date_prec": p.Date, "date": c.Date, "r_indice_log": np.log(c.Close / p.Close),
                       "r_indice_simple": c.Close / p.Close - 1, "gap": jours_ouvres_entre(p.Date,
c.Date)})
ind_paires = pd.DataFrame(ind_paires)
```

```
Indice sur la fenêtre : 986 lignes ; 2023-01-02 → 2026-10-15
Doublons : 0 | clôtures manquantes : 0
Clôtures identiques à la séance précédente (indice) : 49 — à noter : la clôture du 15/10/2026 répète
celle du 14/10/2026
```

H.2 Rapprochement des séries sur des intervalles identiques

Les rendements sont appariés sur les mêmes paires de dates consécutives. Les dates non appariées sont listées.

```
cles_solibra = set(zip(R_Q.date_prec, R_Q.date))
cles_indice = set(zip(ind_paires[ind_paires.gap == 0].date_prec, ind_paires[ind_paires.gap == 0].date))
commun = sorted(cles_solibra & cles_indice)
apparies = (R_Q.set_index(["date_prec", "date"]).join(ind_paires.set_index(["date_prec", "date"]),
how="inner")
            .reset_index())
apparies = apparies[~apparies.classe.str.startswith("bloqué")]
print(f"Rendements appariés (SOLIBRA quotidien admissible et indice sans lacune) : {len(apparies)}")
dates_solibra = set(obs.Date)
dates_indice = set(ind.Date)
non_app_solibra = sorted(d for d in dates_solibra - dates_indice)
non_app_indice = sorted(d for d in dates_indice - dates_solibra)
print(f"Dates SOLIBRA absentes de l'indice : {len(non_app_solibra)} ; dates de l'indice absentes de
SOLIBRA : {len(non_app_indice)}")
pd.DataFrame({"date": [d.date().isoformat() for d in non_app_solibra], "serie": "SOLIBRA seul"}).to_csv(
    os.path.join(SORTIE, "tables", "dates_non_appariees.csv"), index=False)
pd.concat([pd.DataFrame({"date": [d.date().isoformat() for d in non_app_solibra], "serie": "SOLIBRA
seul"}),
           pd.DataFrame({"date": [d.date().isoformat() for d in non_app_indice], "serie": "indice
seul"})]).to_csv(
    os.path.join(SORTIE, "tables", "dates_non_appariees.csv"), index=False)
```

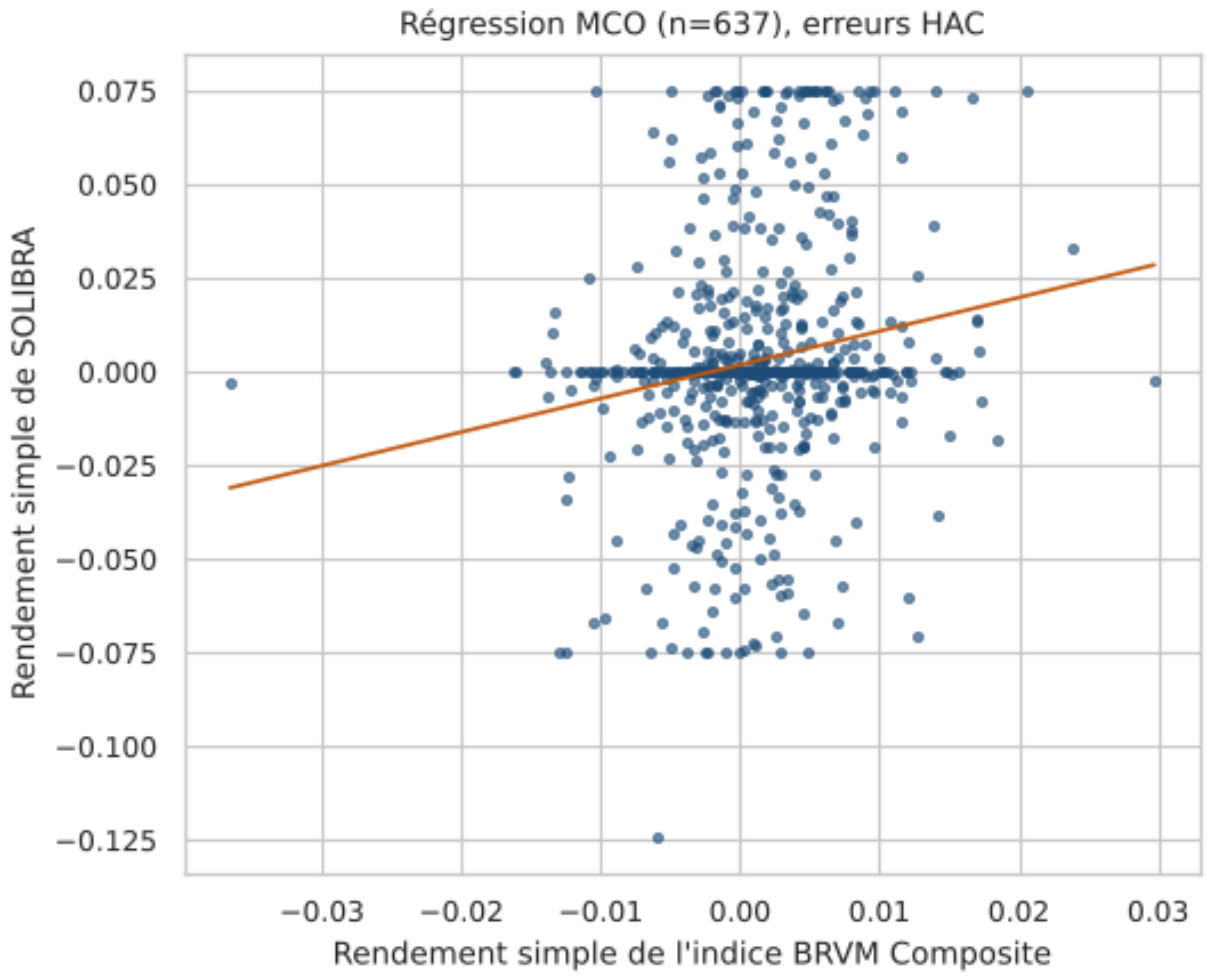
```
Rendements appariés (SOLIBRA quotidien admissible et indice sans lacune) : 637
Dates SOLIBRA absentes de l'indice : 0 ; dates de l'indice absentes de SOLIBRA : 265
```

H.3 Performances, corrélations et régression

Les corrélations sont mesurées par Pearson et Spearman. La régression utilise des erreurs standards HAC.

```
if len(apparies) >= 30:
    pear = stats.pearsonr(apparies.r_simple, apparies.r_indice_simple)
    spear = stats.spearmanr(apparies.r_simple, apparies.r_indice_simple)
    lags_hac = int(np.floor(4 * (len(apparies) / 100) ** (2 / 9)))
    X = sm.add_constant(apparies.r_indice_simple.values)
    modele = sm.OLS(apparies.r_simple.values, X).fit(cov_type="HAC", cov_kwds={"maxlags": lags_hac})
    ic = modele.conf_int(alpha=0.05)
    reg = pd.DataFrame([{
        "n": len(apparies), "pearson": pear.statistic, "pearson_p": pear.pvalue,
        "spearman": spear.statistic, "spearman_p": spear.pvalue,
        "beta_marche": modele.params[1], "beta_IC95_bas": ic[1, 0], "beta_IC95_haut": ic[1, 1],
"beta_p_HAC": modele.pvalues[1],
        "constante_non_alpha": modele.params[0], "constante_p_HAC": modele.pvalues[0],
        "R2": modele.rsquared, "HAC_maxlags": lags_hac,
    }])
    reg.to_csv(os.path.join(SORTIE, "tables", "regression_marche.csv"), index=False)
    pd.set_option("display.float_format", lambda v: f"{v:,.5f}")
    print(reg.T.to_string())
    pd.set_option("display.float_format", None)
    fig, ax = plt.subplots(figsize=(5.5, 4.5))
    ax.scatter(apparies.r_indice_simple, apparies.r_simple, s=8, alpha=0.6, color="#1f4e79")
    xx = np.linspace(apparies.r_indice_simple.min(), apparies.r_indice_simple.max(), 50)
    ax.plot(xx, modele.params[0] + modele.params[1] * xx, color="#c55a11", lw=1.2)
    ax.set_xlabel("Rendement simple de l'indice BRVM Composite"); ax.set_ylabel("Rendement simple de
SOLIBRA")
    ax.set_title(f"Régression MCO (n={len(apparies)}), erreurs HAC")
    fig.text(0.01, -0.03, "Note : rendements appariés sur mêmes paires de dates sans lacune ; pas de
causalité inférée.", fontsize=7)
    fig.tight_layout()
    fig.savefig(os.path.join(SORTIE, "figures", "07_regression_indice.png"), dpi=dpi_png,
bbox_inches="tight")
    fig.savefig(os.path.join(SORTIE, "figures", "07_regression_indice.pdf"), bbox_inches="tight")
    plt.show(); plt.close(fig)
else:
    print("Effectif apparié < 30 : régression non estimée")
```

```
                         0
n                  637.00000
pearson              0.17259
pearson_p            0.00001
spearman             0.15785
spearman_p           0.00006
beta_marche          0.89753
beta_IC95_bas        0.44730
beta_IC95_haut       1.34776
beta_p_HAC           0.00009
constante_non_alpha  0.00186
constante_p_HAC      0.12119
R2                   0.02979
HAC_maxlags          6.00000
```



Note : rendements appariés sur mêmes paires de dates sans lacune ; pas de causalité inférée.

H.4 Performance comparée (cours) sur les mêmes bornes

Les performances des cours sont comparées sur les mêmes bornes que le titre.

```
def perf_indice(a, b):
    s = ind[(ind.Date >= a) & (ind.Date <= b)]
    return s.Close.iloc[-1] / s.Close.iloc[0] - 1 if len(s) > 1 else np.nan,
(s.Date.iloc[0].date().isoformat() if len(s) else None), (s.Date.iloc[-1].date().isoformat() if len(s)
else None)

comp = []
for nom, (a, b) in [("S3 post-fractionnement", (SPLIT_DATE, FIN)), ("2025 complet", (pd.Timestamp("2025-
01-01"), pd.Timestamp("2025-12-31")))]:
    s_sol = obs[(obs.Date >= a) & (obs.Date <= b)]
    p_sol = s_sol.Close.iloc[-1] / s_sol.Close.iloc[0] - 1 if len(s_sol) > 1 else np.nan
    p_ind, d0, d1 = perf_indice(a, b)
    comp.append({"fenetre": nom, "perf_SOLIBRA": p_sol, "perf_indice": p_ind, "debut_indice": d0,
"fin_indice": d1})
comp_tab = pd.DataFrame(comp)
comp_tab.to_csv(os.path.join(SORTIE, "tables", "comparaison_performance_indice.csv"), index=False)
print(comp_tab.to_string(index=False))

div_pos = dividendes[dividendes.exercice.isin(["2024", "2025"])].copy()
rt_rows = []
for _, r in div_pos.iterrows():
    ex = pd.Timestamp(r.date_detachement)
    avant = obs[obs.Date < ex].Close
    if len(avant) == 0:
        continue
    p0_date = obs[obs.Date < ex].Date.iloc[-1]
    rt_rows.append({"exercice": r.exercice, "date_detachement": r.date_detachement,
"pre_detachement_date": p0_date.date().isoformat(),
                    "pre_detachement_cours": avant.iloc[-1], "brut_FCFA":
float(r.dividende_brut_FCFA_par_action) if r.dividende_brut_FCFA_par_action else np.nan,
                    "net_FCFA": float(r.dividende_net_FCFA_par_action)})
div_tab = pd.DataFrame(rt_rows)
a0, a1 = pd.Timestamp("2025-01-01"), FIN
s_w = obs[(obs.Date >= a0) & (obs.Date <= a1)]
p_start, p_end = s_w.Close.iloc[0], s_w.Close.iloc[-1]
brut_tot = div_tab.loc[div_tab.date_detachement.between(a0.date().isoformat(), a1.date().isoformat()),
"brut_FCFA"].sum()
net_tot = div_tab.loc[div_tab.date_detachement.between(a0.date().isoformat(), a1.date().isoformat()),
"net_FCFA"].sum()
rt = pd.DataFrame([{
    "fenetre": "2025-01-02 → 2026-10-14 (aucune base indéterminée, aucun événement de capital documenté
dans la fenêtre)",
    "cours_debut": p_start, "cours_fin": p_end,
    "perf_cours": p_end / p_start - 1,
    "rendement_total_brut_indicatif": (p_end + brut_tot) / p_start - 1,
    "rendement_total_net_indicatif": (p_end + net_tot) / p_start - 1,
    "dividendes_bruts_FCFA": brut_tot, "dividendes_nets_FCFA": net_tot,
    "condition": "dividendes documentés uniquement par le dépôt (avis PDF non ouverts) : indicatif",
}])
rt.to_csv(os.path.join(SORTIE, "tables", "rendement_total_indicatif.csv"), index=False)
print(rt.T.to_string())
print("\nSharpe : NON CALCULÉ — aucun taux sans risque documenté (TAUX_SANS_RISQUE = None).")
```

```
               fenetre  perf_SOLIBRA  perf_indice debut_indice fin_indice
S3 post-fractionnement      2.480638     1.035078   2024-09-30 2026-10-15
          2025 complet      1.223077     0.252627   2025-01-01 2025-12-31

0
fenetre                          2025-01-02 → 2026-10-14 (aucune base indéterminée, aucun événement de
capital documenté dans la fenêtre)
cours_debut
13000
cours_fin
38200
perf_cours
1.938462
rendement_total_brut_indicatif
2.195923
```

```
rendement_total_net_indicatif
2.165028
dividendes_bruts_FCFA
3347.0
dividendes_nets_FCFA
2945.36
condition                                                dividendes documentés uniquement par le dépôt
(avis PDF non ouverts) : indicatif

Sharpe : NON CALCULÉ — aucun taux sans risque documenté (TAUX_SANS_RISQUE = None).
```